JavaScript should come with batteries

Who am I?

Luca Casonato

Software Engineer at Deno Land Inc

Working on Deno Deploy, the Deno CLI, and Fresh

Delegate at TC39

Contributor to WHATWG and W3C specifications

Co-chair of WinterCG

Today’s premise

JavaScript is bare bones. This is bad!

JavaScript has no extensive standard library or default tooling.

Task	JavaScript
Formatting	`prettier`
Linting	`eslint`
Testing	`mocha` *or* `jest` *or* `chai`
Types	`tsc`
Types but fast	`esbuild` *or* swc
Benchmarking	`mitata` *or* `benchmarking`
Documentation	`typedoc`
YAML parser	`yaml` *or* `js-yaml`
Async utils	`p-map` *and* `p-retry`

Every tool and API you need is one more package to install.

There is a lot of *different* tools here.

And you want to use these together?

`eslint` + TypeScript

❌ No TypeScript support out of the box.

First install two more dependencies:

```
@typescript-eslint/parser @typescript-eslint/eslint-plugin
```

Then add a bunch of config to `.eslintrc`:

```
module.exports = {
  extends: ["eslint:recommended", "plugin:@typescript-eslint/recommended"],
  parser: "@typescript-eslint/parser",
  plugins: ["@typescript-eslint"],
  root: true,
};
```

`eslint` + JSX

❌ No JSX support out of the box.

Can be added via plugins (there are multiple).

`eslint` + ESM

✅ Support for parsing and linting ESM.

❌ Can’t use ESM for config file in version 8 (which is the most widely used version).

✅ Support for ESM config file in version 9.

`jest` + TypeScript

❌ No TypeScript support out of the box.

Install a dependency:

`@babel/preset-typescript`

Update your `babel.config.js` file:

```
module.exports = {
  presets: [
    ["@babel/preset-env", { targets: { node: "current" } }],
    "@babel/preset-typescript",
  ],
};
```

`jest` + JSX

❌ No JSX support out of the box.

Install a dependency:

`@babel/preset-react`

Update your `babel.config.js` file:

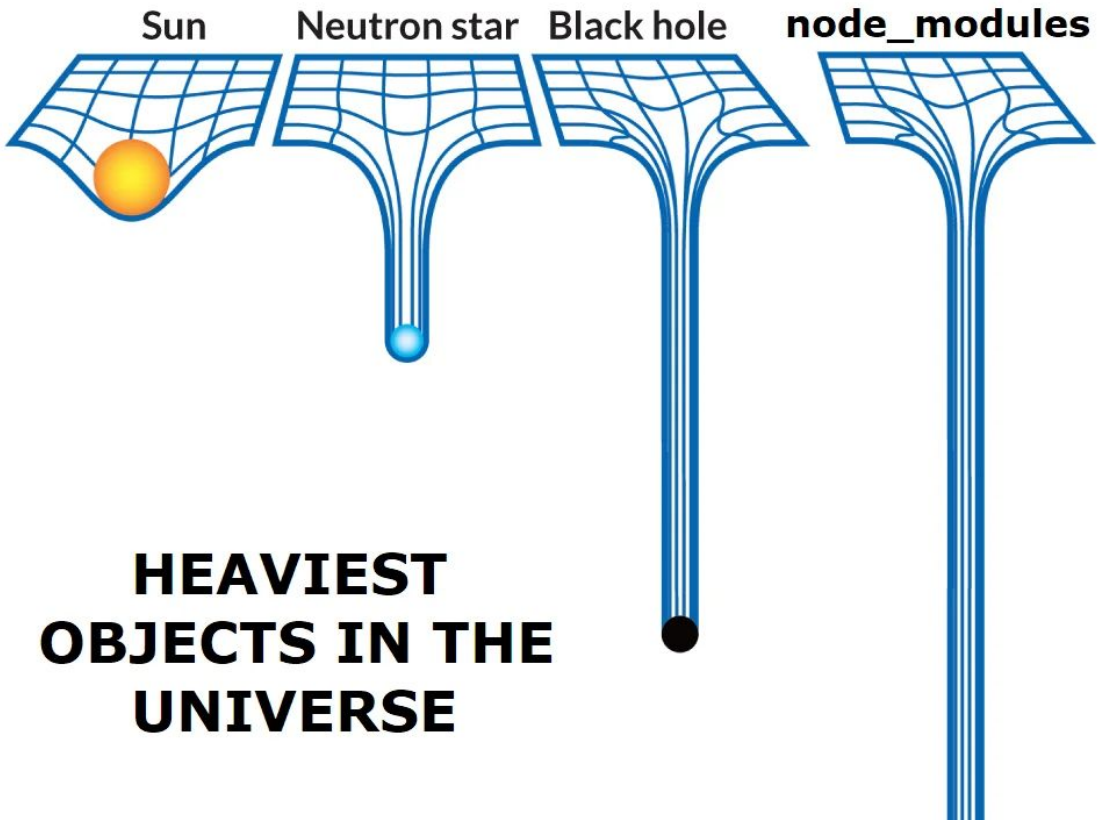
```
module.exports = {
  presets: [
    ["@babel/preset-env", { targets: { node: "current" } }],
    ["@babel/preset-react", { runtime: "automatic" }],
  ],
};
```

`jest` + ESM

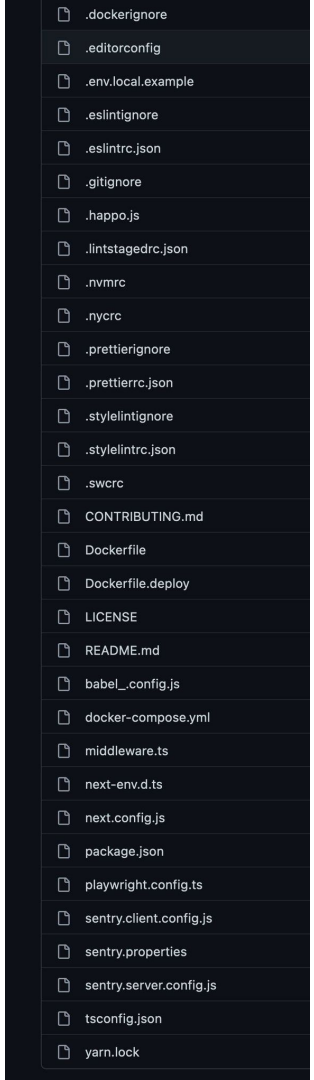
❌ No ESM support out of the box.
❌ No ESM support in the Jest config file.
❌ No ESM support in the Babel config file. Babel supports ESM, but the way Jest integrates Babel prevents it from working in Jest.

Experimental support, requires `--experimental-vm-modules`. Also requires two config file changes, one to Jest config, one to Babel config.

Your node_modules quickly looks like this:



And you need a config file for every tool.



1 project: 30 config files

A tale of blood, sweat, and many wasted hours.

How do we solve this?

How do other ecosystems solve this?

How do other languages do this?

Task	Rust	Go
Formatting	cargo fmt	go fmt
Linting	cargo clippy	go vet
Testing	cargo test	go test
Benchmarking	cargo bench	go bench
Documentation	cargo doc	go doc

This is great!

Get started without spending any time on configuration.

No discussions on using x vs y vs z. Everyone uses the same tooling.

Switching between companies / teams does not mean switching tooling!

So let’s do something about this!

But who to do it?

In 2010, Ryan Dahl released Node.js.

Ryan then went on to use Go for a couple years.

Back in 2018 he came back to JS, and reflected on Node, concluding this was a problem worth revisiting.

In May 2020, 10 years after the original release of Node, we released Deno to fix Node's problems.

Deno: Next-generation JavaScript Runtime

- Incredibly simple to get started
- Secure by default
- Native support for TypeScript and JSX
- Testing, linting, formatting, and more out of the box
- Web Standard APIs
- NEW Backwards compatible with Node.js and npm

Trusted by:

slack netlify GitHub supabase

Simplicity starts at installation

macOS and Linux

```
curl -fsSL https://deno.land/x/install/install.sh | sh
```

Windows

```
irm https://deno.land/install.ps1 | iex
```

Also available on `brew`, `scoop`, `choco`, `nix`, `pacman`, `asdf`, and more

Demo: The Basics

Demo: I/O

Permissions

Just like browsers, Deno prevents dangerous I/O by default.

You have to explicitly opt in.

Either accept prompts, or pass flags upfront.

Demo: Web Standard APIs

Deno builds on Web Standard APIs

Deno uses Web APIs everywhere.

This helps you get going quickly. Learn once, use in browser & on the server.

Deno collaborates in standards groups to make this work well:

WinterCG

Web-interoperable Runtimes Community Group

Import and run from URLs

Deno is a browser for the CLI.

Just like browsers, Deno can import & run JavaScript from URLs.

Demo: Run from URLs

Importing NPM packages

Deno can import and run NPM packages out of the box.

Deno does not require package.json or node_modules.

Deno is runtime and package manager in one.

Demo: Cowsay

Demo: Express

deno lint

deno fmt

Deno KV

- A key-value database built right into Deno
- ACID compliant
- Built for serverless applications
- Task queues built-in
- Key watching (coming soon)

No setup, no install, no import. Just use it.

Demo: Counter with Deno KV

Quick tangent: how does Deno make money?

Everything I have shown you so far is free, open source, and MIT licensed.

Deno Deploy: the serverless JavaScript hosting platform.

- Works great with Deno
- Supports NPM modules
- Global distribution (edge) to reduce latency
- Built in persistence (Deno KV)
- Deploy from GitHub
- Subhosting offering to allow integration into your own product

Build & test locally, deploy globally

If it works locally, you can be confident it works too when deployed.

Same runtime, same APIs, but backed by systems built for scale.

Example:

- Deno KV is locally backed by SQLite
- When deployed, it’s backed by FoundationDB (the DB that powers iCloud)

Demo: deploying our app

Recap

In a matter of 20 minutes:

- We started a new project from scratch.
  - with formatting
  - with linting
  - with a database
  - using NPM dependencies
- We deployed it to the cloud.

We used no tooling other than the tooling that comes with Deno.

We used 16% of Deno's tools today.

```
Commands:
  run          Run a JavaScript or TypeScript program
  bench        Run benchmarks
  bundle       Bundle module and dependencies into single file
  cache        Cache the dependencies
  check        Type-check the dependencies
  compile      Compile the script into a self contained executable
  completions  Generate shell completions
  coverage     Print coverage reports
  doc          Show documentation for a module
  eval         Eval script
  fmt          Format source files
  init         Initialize a new project
  info         Show info about cache or info related to source file
  install      Install script as an executable
  jupyter      Deno kernel for Jupyter notebooks
  uninstall    Uninstall a script previously installed with deno install
  lsp          Start the language server
  lint         Lint source files
  repl         Read Eval Print Loop
  task         Run a task defined in the configuration file
  test         Run tests
  types        Print runtime TypeScript declarations
  upgrade      Upgrade deno executable to given version
  vendor       Vendor remote modules into a local directory
  help         Print this message or the help of the given subcommand(s)
```

That’s “batteries included” for you

All programming languages should work like this.

“Batteries included” significantly increases productivity.

But more importantly, it makes programming more fun.

Thanks for listening!

Check out Deno: https://deno.com

Try out Deno Deploy: https://deno.com/deploy

Join the Discord: https://discord.gg/deno